

TRABAJO PRÁCTICO FINAL

Módulo 1 — Ingeniería de Datos con Python y Pandas

DataNova Argentina S.A.

Contexto del caso

Sos el Data Scientist de DataNova Argentina S.A., una empresa de comercio electrónico B2B que distribuye productos de tecnología, mobiliario y suministros de oficina a empresas en Estados Unidos, Alemania, España y Rumania.

El área de ventas te pasó tres archivos Excel exportados de su base de datos SQL porque el servidor está en mantenimiento. Necesitan que analices la información y respondas una serie de preguntas de negocio antes del cierre de trimestre.

Además, el equipo de estrategia te pidió enriquecer el análisis con datos externos sobre los países de los clientes: población, PIB y salario promedio.

Fuentes de datos

Archivos locales (adjuntos al TP)

```
#para leer archivos excel pd.read_excel('nombre_archivo')
```

- Ventas.xlsx → df_ventas
- Clientes.xlsx → df_clientes
- Productos.xlsx → df_productos

Sitios WEB:

- Población (Wikipedia) — tabla índice 0
- `url_pob`=https://es.wikipedia.org/wiki/Anexo:Pa%C3%ADses_y_territorios_dependientes_por_poblaci%C3%B3n
- PIB (Worldometers) — tabla índice 0
- `url_pib` = '<https://www.worldometers.info/gdp/gdp-by-country/>'
- Salario promedio (Wikipedia) — tabla índice 2

- `url_rpc = 'https://en.wikipedia.org/wiki/List_of_countries_by_average_wage'`

Fuentes web (web scraping con `pd.read_html`)

#para leer archivos html `pd.read_html('nombre_archivo')`

- Población: Wikipedia → `df_pob` (tabla índice 0)
- PIB: Worldometers → `df_pib` (tabla índice 0)
- Salario promedio: Wikipedia → `df_rpc` (tabla índice 2)

PASO 1 — Importación y normalización de datos

Importar librerías

Ejemplo de lectura de tablas web de la tabla Población

Carga de datos externos (web scraping con `pd.read_html`)

`pd.read_html()` devuelve una **lista de DataFrames**, uno por cada tabla `<table>` que encuentra en la página. Usamos el índice correcto de cada fuente.

```
headers = {
    "User-Agent": "Mozilla/5.0" #simula un navegador real
}
url_pob = "https://en.wikipedia.org/wiki/List_of_countries_and_dependencies_by_population"
...
html_pob = requests.get(url_pob, headers=headers).text # Hace la petición HTTP
...
df_pob = pd.read_html(StringIO(html_pob))[0]
#Busca tablas HTML automáticamente y las convierte en DataFrames, en este caso el índice [0]
indica la primera tabla
...
```

Normalización de columnas: antes de cualquier análisis, normalizá los nombres de todas las columnas a minúsculas, sin acentos y con guión bajo como separador (`snake_case`). Esto aplica a los 3 Excel y a las 3 fuentes web.

Ayuda:

Usá `unicodedata.normalize('NFD')` para eliminar acentos y `re.sub` para reemplazar caracteres no alfanuméricos por guión bajo.
Puedes definir una función:

```
def normalizar_columna(col):
    """Convierte nombres de columnas a snake_case sin acentos."""
```

```
col = str(col)
col = col.lower()
col = unicodedata.normalize('NFD', col)
col = col.encode('ascii', 'ignore').decode('utf-8')
col = re.sub(r'^a-z0-9+', '_', col)
col = col.strip('_')
return col
```

PREGUNTA 1: ¿Cuáles son las dimensiones (filas × columnas) de cada uno de los 6 dataframes?

Resultado:

PREGUNTA 2: ¿Existen valores NaN en los dataframes de ventas, clientes y productos? Mostrá el conteo por columna para cada uno.

Resultado:

PASO 2 — Limpieza de inconsistencias

Detectá y corregí inconsistencias en `df_ventas` y `df_clientes`.

2.1 Borrar registros sin fecha de compra

PREGUNTA 3: ¿Cuántos registros se eliminaron de `df_ventas` por tener Fecha Compra vacía?

Resultado:

2.2 Borrar registros duplicados

PREGUNTA 4: ¿Existían registros duplicados en `df_ventas` o `df_productos`? ¿Cuántos se eliminaron?

Resultado:

2.3 Inconsistencia en datos categóricos

PREGUNTA 5: ¿Qué valor incorrecto encontraste en la columna `segmento` de `df_clientes`? Corregilo usando `map()` y mostrá el resultado con `value_counts()`.

Resultado:

Ayuda: `map()` con un diccionario es la herramienta correcta para reemplazar valores específicos.

2.4 Conversión de tipos numéricos

PREGUNTA 6: La columna `precio_unitario` llegó como string con coma decimal. Convertila a float. ¿Qué operaciones fueron necesarias?

Resultado:

2.5 Conversión de fechas

PREGUNTA 7: La columna fecha_compra y fecha_envio llegó como object. Convertila a dateTime. ¿Qué operaciones fueron necesarias?

Resultado:

PASO 3 — Limpieza estadística y outliers

PREGUNTA 8: Aplicá describe() sobre df_ventas. ¿Cuál es la mediana del precio_unitario? ¿Y el percentil 75 de cantidad?

Resultado:

La empresa tiene una restricción operativa: ningún pedido puede tener más de 13 unidades.

PREGUNTA 9: ¿Cuántos pedidos tienen más de 13 unidades? Eliminá esos registros y mostrá las nuevas dimensiones de df_ventas.

Resultado:

PREGUNTA 10: Imputá, usando fillna, los registros sin fecha_envio usando: fecha_compra + mediana (usar median()) del plazo de entrega de los pedidos que sí la tienen. ¿Cuántos registros se imputaron?

Resultado:

Ayuda: La diferencia entre dos columnas datetime da timedelta. La mediana (median()) de esos timedeltas se puede sumar directamente a una fecha.

PREGUNTA 11: Usá pd.cut() para clasificar los pedidos: 'Económico' (0-100), 'Estándar' (100-500), 'Premium' (más de 500). Mostrá la distribución con value_counts().

Resultado:

PREGUNTA 12: Usá pd.qcut() para dividir en 4 cuartiles. ¿Los cuartiles coinciden con los rangos del ejercicio anterior? ¿Por qué?

Resultado:

PASO 3b — Carga a base de datos y lectura con SQL

Con `df_ventas` ya limpio, lo persistimos en la base de datos usando `to_sql()`. Este es el flujo real de un pipeline: los datos crudos se limpian en pandas y se almacenan en la BD como fuente de verdad.

Conexión a la base de datos

Usá la opción que corresponda a tu entorno. El código del análisis posterior es el mismo para las tres.

- Opción A — Local (XAMPP / WAMP):
`mysql+pymysql://root:@127.0.0.1:3306/ventas_db`
- Opción B — Railway: credenciales de tu proyecto MySQL en `railway.app`

PREGUNTA 13: Cargá `df_ventas` limpio a la base de datos en una tabla llamada 'ventas' usando `to_sql()`. ¿Cuántos registros se cargaron?

Resultado:

Ayuda: Usá `if_exists='replace'` para reemplazar si ya existe la tabla, e `index=False` para no guardar el índice de pandas. Eliminá las columnas auxiliares (`rango_precio`, `cuartil`) antes de cargar.

PREGUNTA 14: Leé los datos de vuelta con `read_sql()` usando una query que traiga solo las ventas del año 2020 en adelante (`WHERE YEAR(fecha_compra) >= 2020`). ¿Cuántas filas trae la query vs el total cargado?

Resultado:

Ayuda: Esto es SQL pushdown: el servidor filtra antes de enviar los datos. Con tablas de millones de filas la diferencia de performance es enorme.

PREGUNTA 15: Resolvé esta misma pregunta directamente en SQL: ¿cuántos pedidos se registraron por año? Usá GROUP BY en la query y pd.read_sql() para traer el resultado.

Resultado:

PASO 4 — Consolidación y análisis

Combiná el DataFrame leído desde la BD con df_clientes y df_productos para construir df_global.

PREGUNTA 16: Combiná df_ventas_sql, df_clientes y df_productos en df_global con INNER JOIN. ¿Cuántas filas tiene el DataFrame resultante?

Resultado:

PREGUNTA 17: Creá las columnas 'ingreso' ($\text{precio_unitario} \times \text{cantidad}$) y 'beneficio' ($\text{ingreso} - \text{coste_produccion} \times \text{cantidad}$). ¿Cuál es la categoría con mayor beneficio total?

Resultado:

Ayuda: groupby() + agg() para calcular el beneficio total por categoría.

PREGUNTA 18: Usá `transform()` para agregar a `df_global` una columna 'beneficio_medio_categoria'. ¿Cuántos pedidos superan la media de su categoría?

Resultado:

Ayuda:

1. Calcular beneficio medio por categoría
2. Agregar el resultado a cada fila usando `transform()`
3. Comparar beneficio vs promedio

PREGUNTA 19: Creá una `pivot_table` con el beneficio total por país (filas) y categoría (columnas). Incluí totales con `margins=True`.

Resultado:

PREGUNTA 20: La empresa evalúa abrir un centro de distribución. Calculá la diferencia promedio en días entre `fecha_envio` y `fecha_compra` por país. ¿En qué país los tiempos son más largos?

Resultado:

PASO 5 — Reporte

PREGUNTA 21: Exportá 'Reporte_DataNova.xlsx' con dos pestañas: 'Beneficio_por_pais_cat' con la pivot_table del ejercicio 18, y 'Top_productos' con los 10 productos de mayor beneficio total.

Resultado:

Criterios de evaluación

Criterio	Puntaje
Correctitud de los resultados	30 pts
Calidad del código (claridad, nombres de variables, comentarios)	25 pts
Uso de herramientas específicas (map, transform, pivot_table, cut/qcut)	25 pts
Pipeline BD: to_sql, read_sql y pushdown	20 pts
TOTAL	100 pts

Entregá un único notebook .ipynb con el nombre TP_Final_Apellido_Nombre.ipynb. Debe poder ejecutarse de principio a fin sin errores.